Numerical Methods for Non-Linear Mechanics

Viet Anh Quach

1 Introduction

This short report present the code that is assign for the subject "Numerical Methods for Non-Linear Mechanics" of the Master Program for Civil Engineering, taught by professor Stafano-Dal Pont at Laboratory 3SR, University Grenoble Alpes, France. The objective of the course is studying the non-linear problems in mechanics, caused by both the geometric and material behavior. 2 main problems are taken into account: A simplified truss - the console problem in 2D and a generalized Truss problem in 3D, therefore the remaining of this report are divided into two sections of the former subject. The main code is actually written in `C++`, with libraries mostly built from scratch. They have been uploaded to `Github`, the link to download is provided in Appendix A. Furthermore, the content Finite Element Method is introduced also in the course, hence FEM1D's script is included, but the student has not yet completed it for FEM2D. The plotting and schematic diagram were generated by `Asymptote` - a vector graphic language with C-like features. The whole purpose is to learn so any corrections and comments are highly appreciated. I would like to give my thanks toward professor for letting me, even though being a competence-lacking PhD student, could join this program to learn. Personally I really appreciated and enjoyed the class.

2 Theory

The following problem was fully described in lecture course [1] of professor DAL-PONT. This section just represent it but with the particular implemented scheme using within the following codes and the reasons behind these choice.

2.1 Console

We begin with the most simplified problem which the whole geometry is described in fig. 1. Briefly speaking, the simple truss structure is in 2 dimensions, composed by 3 nodes linked by 2 bars only. External force F applied on the node C, producing an angel θ with the vertical axis, causing the displacement of the bars and nodes. Gravity is not taken into account. In this case, vector $\vec{x}$ representing position of node C, is considered as the main unknown, owing to the fact that it is the only free node and fully represent the equilibrium position of the system. $\vec{e}^b$ is the unit vector of the corresponding bar b, which is calculated by the position vector of the node where the bar b begins ($\vec{x}^B$) and

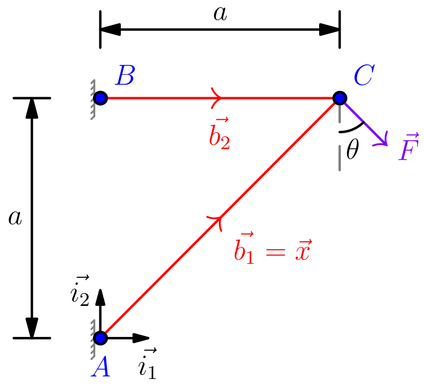


Figure 1: Two elements truss: console 2D

ends $(\vec{x}^E)$:

$$\vec{b} = \vec{x}^{E(b)} - \vec{x}^{O(b)} \tag{2.1a}$$

$$l^b = \|\vec{b}\| \tag{2.1b}$$

$$\vec{e}^b = \frac{\vec{b}}{l^b} \tag{2.1c}$$

In truss elements, the only internal effort considered is the axial stress and further, also is a constant. To satisfy the nodal equilibrium, at position C the sum of the internal force (left-hand side) and the external force (right-hand side) must be balanced:

$$\sum_{b=1}^{2} N^b = N^1 + N^2 = F \tag{2.2}$$

which N^b denotes this component that the right part exerts on the left part of bar b, which is assumed to depend only on the length of bar l^b via the constitutive law:

$$N^b = \mathcal{A}^b(l^b) \tag{2.3}$$

In case of large displacement, this relation is no longer linear, as the bar's vectors are changed accordingly with the new position of the nodes, in our case is particularly node C. The following elastic constitutive laws may be considered, so that the tension N^b taken into account only on the bar length variation:

- Linear law:

$$\mathcal{A}^b(l^b) = \alpha^b \frac{l^b - l_0^b}{l_0^b} \tag{2.4}$$

- Saint-Venant Kirchhoff law:

$$\mathcal{A}^b(l^b) = \alpha^b \frac{(l^b)^2 - (l_0^b)^2}{2(l_0^b)^2} \tag{2.5}$$

- Logarithmic law:

$$\mathcal{A}^b(l^b) = \alpha^b ln(\frac{l^b}{l_0^b}) \tag{2.6}$$

Hence, in general form, the problem of the console could be written as follows:

$$\begin{aligned}
&\text{Given the force } \vec{\mathcal{F}},\\
&\text{Find the position } \vec{x} \text{ of node C such that,}\\
&-\mathcal{A}^1(l^1(\vec{x}))\hat{e}^1(\vec{x}) - \mathcal{A}^2(l^2(\vec{x}))\hat{e}^2(\vec{x}) + \vec{\mathcal{F}} = \vec{0}
\end{aligned} \tag{2.7}$$

This problem is solved by Newton's iterative method in vector form for solving systems of equations. Thanks to first-order Taylor expansion again, we obtain the development of sum of the force, which should be approaching 0, at iteration `k+1` (or in another word, at when node **C** would take an increment in position into $\vec{x} + \delta\vec{x}$):

$$\vec{\mathcal{F}}(\vec{x} + \delta\vec{x}) = \vec{\mathcal{F}}(\vec{x}) + \left(\nabla\vec{\mathcal{F}}(\vec{x})\right) @\delta\vec{x} = 0 \tag{2.8}$$

where the Jacobian matrix $\nabla\vec{\mathcal{F}}(\vec{x})$ is described in the following equation using Einstein's sum notation:

$$\nabla\vec{\mathcal{F}}(\vec{x}) = -\frac{d\mathcal{A}^i}{dl^i}\left(\vec{e}^i(\vec{x}) \otimes \vec{e}^i(\vec{x})\right) - \frac{\mathcal{A}^i\left(l^i(\vec{x})\right)}{l^i(\vec{x})}\left(\mathbb{I} - \vec{e}^i(\vec{x}) \otimes \vec{e}^i(\vec{x})\right) \tag{2.9}$$

Finally, we can solve the system using a single linear vector equation, which is suitable to be implemented into computational calculating:

$$\left(\nabla\vec{\mathcal{F}}(\vec{x}^k)\right) @\delta\vec{x}^k = -\vec{\mathcal{F}}(\vec{x}^k) \tag{2.10}$$

Solving the linear system get us the new value of δx, so we could update the position for the next iteration by:

$$\vec{x}^{(k+1)} = \vec{x}^{(k)} + \delta\vec{x}^{(k)} \tag{2.11}$$

The method requires a stopping criterion: eq. (2.12) is chosen to be implemented in the code, as it satisfies the force equilibrium condition at nodes of the problem.

$$\left|f\left(x^{(k)}\right)\right| < \epsilon \tag{2.12}$$

where ϵ is a number small enough to be considered as 0, relatively to the problem-solving. Another substitute approach is stopping at the iteration where the displacement is negligible, which could be taken into consideration:

$$\left|x^{k+1} - x^k\right| < \epsilon \tag{2.13}$$

2.2 Truss 3D

The 3D truss problem is a problem enhanced from the previous one, which is in 3 dimensions and the numbers of bars could be as numerous as required. Similarly, in the previous problem, each bar's geometry is fully accessible through the position of nodes. Their vectors could be determined totally similar in the previous problem via eq. (2.1). Referencing to eq. (2.11), instead of δx, we determine $\vec{u}^n$ as the displacement of node n between the initial configuration and the deformed one. Defining the set of bars is $\mathcal{B}$, while the set of nodes is $\mathcal{N} = \mathcal{N}_{\mathcal{I}} \cup \mathcal{N}_{\mathcal{F}}$, where $\mathcal{N}_I$ signifies the set of nodes which has its position (displacement) imposed, and $\mathcal{N}_F$ is the set of nodes that is applied with external forces, including the reactions. The problem of the truss system can be stated:

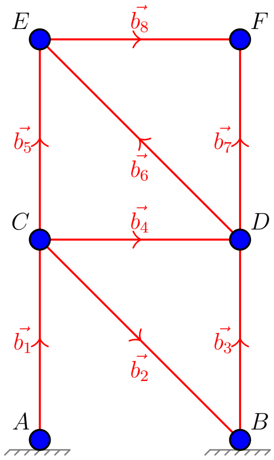


Figure 2: 3D truss

$$\begin{aligned} &\text{Given the external force } \vec{F}^{e/n}, n \in \mathcal{N}_F \text{ and } \vec{U}^n, n \in \mathcal{N}_I \\ &\text{find the displacement vector } \vec{u}^n, n \in \mathcal{N} \text{ so that } \vec{u}^n = \vec{U}^n, n \in \mathcal{N}_I \text{ and} \\ &\forall \vec{v}^n \text{ such that } \vec{v}^n = 0, n \in \mathcal{N}_I \\ &-\sum_{b \in \mathcal{B}} N^b \vec{e}^b \left(\vec{v}^{E(b)} - \vec{v}^{O(b)}\right) + \sum_{n \in \mathcal{N}_F} \vec{F}^{e/n} \vec{v}^n = 0 \\ &\text{where:} \quad N^b = \mathcal{A}^b\left(l^b\right) \end{aligned} \tag{2.14}$$

The reactions $\vec{R}^n$ at imposed nodes are not explicitly solved but are implicitly enforced through the penalty method. After convergence, reactions can be computed as:

$$\vec{R}^n = \vec{F}^{int/n} - \vec{F}^{ext/n}, \quad n \in \mathcal{N}_I \tag{2.15}$$

where $\vec{F}^{int/n}$ is the internal force at node n from connected bars. The above equation could be expressed in matrix notation as:

$$\forall \underline{\mathcal{V}}, \underline{\mathcal{V}}^t \underline{\underline{\mathcal{K}}} \underline{\mathcal{U}} = \underline{\mathcal{V}}^t \underline{\mathcal{F}} \tag{2.16}$$

or in the simplest way:

$$\underline{\underline{\mathcal{K}}}\, \underline{\mathcal{U}} = \underline{\mathcal{F}} \tag{2.17}$$

We can easily verify that:

$$\underline{\mathcal{F}} = \begin{bmatrix} F_1^{e/1} \\ F_2^{e/1} \\ F_3^{e/1} \\ \vdots \\ F_1^{e/N} \\ F_2^{e/N} \\ F_3^{e/N} \end{bmatrix} ; \quad \underline{\mathcal{U}} = \begin{bmatrix} u_1^1 \\ u_2^1 \\ u_3^1 \\ \vdots \\ u_1^N \\ u_2^N \\ u_3^N \end{bmatrix} \tag{2.18}$$

Within each component the subscript $k = 1, 2, 3$ stands for the 3 directions in 3D and the superscript $n = 1..N$ determines indices of nodes.

Meanwhile, the global stiffness matrix $\underline{\underline{\mathcal{K}}}$ could be assembled by multiple methods. One of them is by using the elementary application K^b of bar b along with the connectivity matrix $\underline{\underline{C^n}}$ as followed:

$$\underline{\underline{\mathcal{K}}} = \sum_{b \in \mathcal{B}} \left(\underline{\underline{C}}^{E(b)} - \underline{\underline{C}}^{O(b)} \right) \underline{\underline{K}}^b \left(\underline{\underline{C}}^{E(b)} - \underline{\underline{C}}^{O(b)} \right) \tag{2.19}$$

$$\text{where: } \underline{\underline{C^n}} = \left[\begin{array}{ccc|ccc|ccc} 0 & \cdots & 0 & 1 & 0 & 0 & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & 0 & 1 & 0 & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & 0 & 0 & 1 & 0 & \cdots & 0 \end{array} \right] \tag{2.20}$$

$$\underbrace{\qquad\qquad}_{3(n-1)} \quad \underbrace{\qquad}_{3} \quad \underbrace{\qquad\qquad}_{3(N-n)}$$

$$\text{and: } \underline{\underline{K}}^b = \tilde{\mathcal{A}}^b(l^{b(k)})\mathbb{I} + \frac{1}{l^{b(k)}} \frac{d\tilde{\mathcal{A}}^b}{dl} \vec{r}^{b(k)} \otimes \vec{r}^{b(k)}; \tag{2.21}$$

$$\text{denoting } \tilde{\mathcal{A}}^b\left(l^b\right) = \frac{\mathcal{A}^b\left(l^b\right)}{l^b} \tag{2.22}$$

Though this method is easily implemented in coding small problem, at the numerical cost owing to storing and calculating all the huge dimension connectivity matrix. At this point, $\underline{\underline{\mathcal{K}}}$ should be singular i.e. $\det(\underline{\underline{\mathcal{K}}}) = 0$ because the kinematic boundary condition is not yet applied. Two different techniques are usually proposed: the direct method and the approximated method (so-called penalization method). The approximated is obtained by adding $1/\epsilon$ to the main diagonal terms of $\underline{\underline{\mathcal{K}}}$ which corresponds to the nodes of $\mathcal{N}_I$:

$$\underline{\underline{\mathcal{K}}}^{\epsilon}_{ii} = \underline{\underline{\mathcal{K}}}_{ii} + \frac{1}{\epsilon}; i = 3(n-1) + k; n \in \mathcal{N}_I, k = 1, 2, 3 \tag{2.23}$$

$$\mathcal{F}^{\epsilon}_i = \begin{cases} \mathcal{F}^{\epsilon}_i + \frac{1}{\epsilon} U^n_k & \text{for} \quad i = 3(n-1) + k; n \in \mathcal{N}_I, k = 1, 2, 3 \\ \mathcal{F}^D_i & \text{if not} \end{cases} \tag{2.24}$$

After assembling and the injection of kinematic condition, eq. (2.17) is solved until stopping criterion eq. (2.12) is satisfied on the whole set of nodes.

2.3 1D Finite Element Method

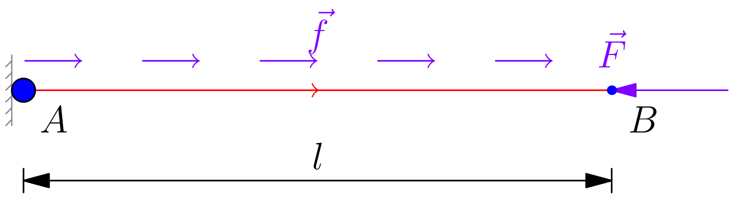


Figure 3: 1D Finite Element

The equilibrium equations of the 1-side fixed bar described in fig. 3, expressed in the strong form (differential form):

$$\frac{dN}{dx} + f = 0; \tag{2.25a}$$
$$N(0) = -R; \tag{2.25b}$$
$$N(l) = F; \tag{2.25c}$$
$$N = EA\frac{du}{dx}; \tag{2.25d}$$
$$u(0) = 0; \tag{2.25e}$$

The weak form is expressed by;

$$\forall v \in V, \int_0^l EA\frac{du}{dx}\frac{dv}{dx}dx - Rv(0) = \int_0^l fvdx + Fv(l) \tag{2.26}$$

where $v \in V_0$ is the virtual field that we can choose such that $v(0) = 0$ to get rid of the reaction part in eq. (2.26). Or rewrite in another linear compacted form:

$$\forall v \in V, a(u, v) + b(R, v) = L(v) \tag{2.27}$$
$$\text{with: } a(u, v) = \int_0^l EA\frac{du}{dx}\frac{dv}{dx}dx; \tag{2.28}$$
$$b(R, v) = -Rv(0) = 0; \tag{2.29}$$
$$L(v) = \int_0^l fvdx + Fv(l) \tag{2.30}$$

The displacement field $u(x)$, similarly $v(x)$ is approximated by discretizing via Galerkin's method:

$$u(x) \approx u^a(x) = U^i N^i;\ \ v(x) \approx v^a(x) = V^i N^i \tag{2.31}$$

Finite Element Method is a Galerkin-like method but with a particular choice of shape function N^i as the low-order hat function via Lagrange's basis.

$$N^i(x_j) = \delta_{ij} = \frac{x - x_j}{x_i - x_j} \tag{2.32}$$

Rewritten eq. (2.26) in tensor notation:

$$\text{Find } \underline{\mathcal{U}} \text{ and } \mathcal{R} \text{ such that:}$$
$$U^1 = 0; \tag{2.33}$$
$$\underline{\underline{\mathcal{K}}}\,\underline{\mathcal{U}} + \mathcal{R} = \underline{\mathcal{F}} \tag{2.34}$$

The boundary condition in this section is applied through direct method, with 2 types of well-known condition: the Dirichlet's condition and Neumann's condition. In order to calculate each element of stiffness matrix $\mathcal{K}_{ij} = EA\int_0^l N_{i,x}N^{j,x}dx$. The numerical integration scheme is Gaussian quadrature (document distributed during the course) with 2 points, values referencing in table 1.

$$\int_{-1}^{1} h(\xi)d\xi = \sum_{i=1}^{N} h(\xi_i)W_i \tag{2.35}$$

After assembly the stiffness matrix, by choosing the suitable virtual field, we could calculate the displacement vector $\underline{\mathcal{U}}$ and then identify the reaction $\mathcal{R}$ afterward.

Table 1: Gauss-Legendre Quadrature Points and Weights in range $[-1,1]$

n	Gauss Points ξ_i	Weights W_i
1	$\{0\}$	$\{2\}$
2	$\left\{-\frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}\right\}$	$\{1,1\}$
3	$\left\{-\sqrt{\frac{3}{5}}, 0, \sqrt{\frac{3}{5}}\right\}$	$\left\{\frac{5}{9}, \frac{8}{9}, \frac{5}{9}\right\}$
4	$\left\{\begin{array}{c}\pm\sqrt{\frac{3}{7}+\frac{2}{7}\sqrt{\frac{6}{5}}},\\ \pm\sqrt{\frac{3}{7}-\frac{2}{7}\sqrt{\frac{6}{5}}}\end{array}\right\}$	$\left\{\begin{array}{c}\frac{18-\sqrt{30}}{36},\\ \frac{18+\sqrt{30}}{36}\end{array}\right\}$

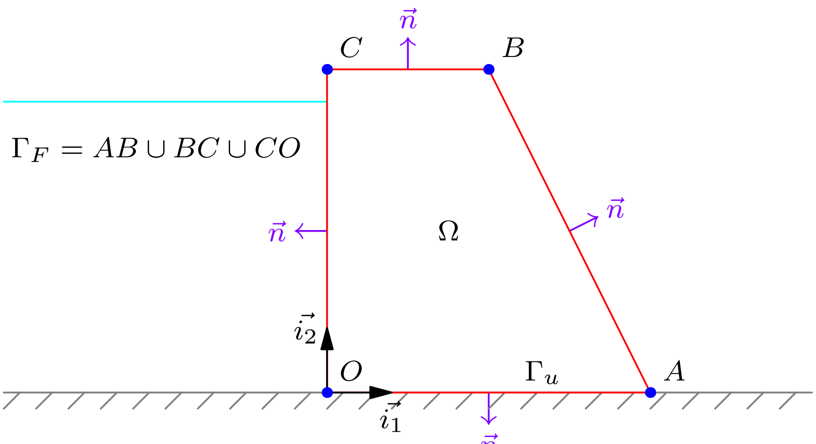


Figure 4: Deformed console

2.4 2D Finite Element Method

$$\forall \vec{x} \in \Omega, \mathrm{div}\sigma + \vec{f} = 0; \tag{2.36}$$
$$\forall \vec{x} \in \Omega, \sigma = \lambda \mathrm{tr}\epsilon(\vec{u})\mathbb{I} + 2\mu\epsilon(\vec{u}); \tag{2.37}$$
$$\forall \vec{x} \in \Gamma_u, \vec{u} = \vec{U}; \tag{2.38}$$
$$\forall \vec{x} \in \Gamma_F, \sigma@\vec{n} = \vec{F}; \tag{2.39}$$
$$\forall \vec{x} \in \Gamma_u, \sigma@\vec{n} = \vec{R}; \tag{2.40}$$

weak form:

$$\text{Find } \vec{u} \in V_0 \text{ and } \vec{\mathcal{R}} \text{ such that:}$$
$$\forall \vec{v} \in V_0, \int_\Omega (\lambda \mathrm{tr}\epsilon(\vec{u})\mathrm{tr}\epsilon(\vec{v}) + 2\mu\epsilon(\vec{u}) : \epsilon(\vec{v}))ds - \int_{\Gamma_u} \vec{R}\vec{v}dl = \int_\Omega \vec{f}\vec{v}ds + \int_{\Gamma_F} \vec{F}\vec{v}dl; \tag{2.41}$$

This equation could be expressed in the linear form in the same way as eq. (2.30) and eq. (2.34), then the processing is executed in similar way as in the previous section.

3 Validation

3.1 Console

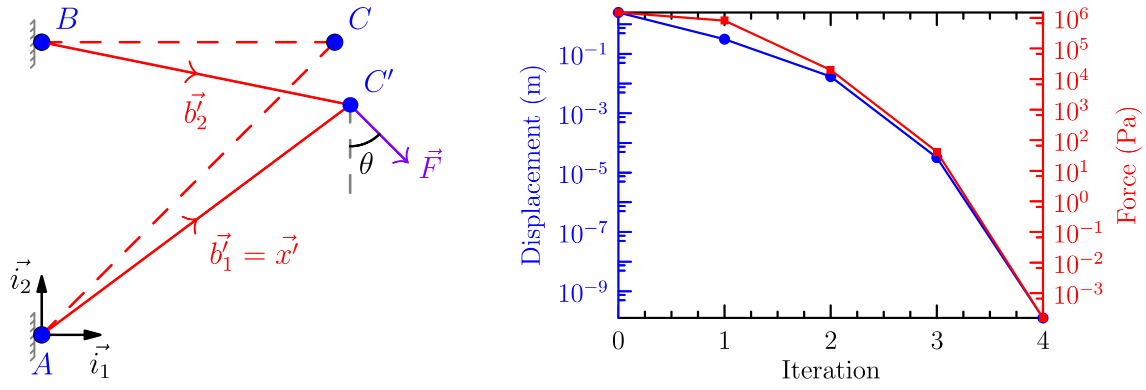
The initial position x_0 is, intuitively, taken by the initial configuration to assure better convergence and avoid unnecessary numbers of iteration. Assume the bar to be rectangle, the input parameters are put in a namespace `modelParameters` within the code, so one could change it accordingly to needs.

```cpp
namespace modelParameters {
// Bar's parameters
double E{25e9}; // Module Young
double a{10.0}; // Distance between node 0 and 1 = bar length

double force{1.5e6};                    // Imposed Force
// Inclined angle between the force with relative to vertical axis
double theta{20};

// Section dimension (rectangle)
double b1{0.02}, h1{0.05};
double b2{0.02}, h2{0.05};

// tolerance for Newton's method
double epsilon{1e-8};

// Choosing non-linear Saint-Venant Kirchhoff law
int law{1};
}
```

The final result obtained by printing to the console (terminal) is:

```
=== 2D CONSOLE PROBLEM ===
Iteration 0: |Fk| = 1.5e+06: |dx| = 2.48569
Iteration 1: |Fk| = 814055: |dx| = 0.312942
Iteration 2: |Fk| = 19706.4: |dx| = 0.0172449
Iteration 3: |Fk| = 42.1839: |dx| = 3.20384e-05
Iteration 4: |Fk| = 0.00015477: |dx| = 1.26595e-10
Solution found at iteraion 5.

Final displacement: [0.526986, -2.14187]
Final position: [10.527, 7.85813]
Time elapsed: 0.000122333 seconds
```

As showed in diagram fig. 6a, under the effect of force $\vec{F}$, node C displaced from position (10,10) to (10.53, 7.86). These values are all justified by calculation manually. At the beginning of iteration number 5, noticing that the sum of forces acting on node C and its displacement has already approached to a value less than $\epsilon = 10^{-10}$, the process stopped, and the final value was taken at the last step, which is iteration number 4. As proved in figure fig. 6b, the rate of convergence (in logarithm proportion) by Newton method is quadratic. Student chose the Saint-Venant Kirchhoff law to present the result, with the intention to capture the non-linearity in both the constitutive law via eq. (2.3) and the large deformation's effect in eq. (2.1).



(a) Deformed console schematic (b) Displacement and sum of forces on node C

Figure 5: Solutions of the 2D console problem

3.2 Truss 3D

The following lines are input parameters of the code:

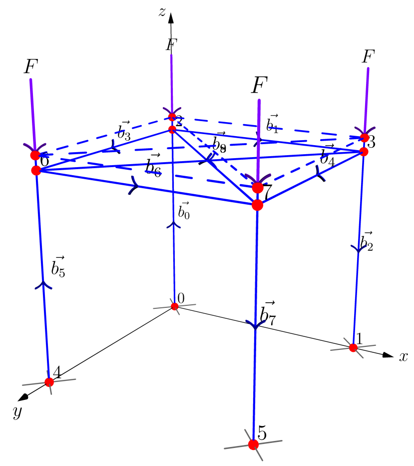
```
// Model Parameters
namespace modelParameters {
// Problem dimension:
constexpr Index d{3};

// Geometry of the truss
constexpr Index nNodes{8}; // number of nodes
constexpr Index nBars{10}; // number of bars

Vector<Vector<double>> nodes{
    {0.0, 0.0, 0.0},      // Node 0
    {10.0, 0.0, 0.0},     // Node 1
    {0.0, 0.0, 10.0},     // Node 2
    {10.0, 0.0, 10.0},    // Node 3
    {0.0, -10.0, 0.0},    // Node 4
    {10.0, -10.0, 0.0},   // Node 5
    {0.0, -10.0, 10.0},   // Node 6
    {10.0, -10.0, 10.0}, // Node 7

};
// Bar connectivity: store node indices for each bar's origin and
   end
Vector<Index> barOrigin{0, 2, 3, 2, 3, 4, 6, 7, 3, 2};
Vector<Index> barEnd{2, 3, 1, 6, 7, 6, 7, 5, 6, 7};

Vector<Index> nodeImposed{0, 1, 4, 5};
Vector<Vector<double>> displacementImposed{
    {0.0, 0.0, 0.0}, {0.0, 0.0, 0.0}, {0.0, 0.0, 0.0}, {0.0, 0.0,
       0.0}};

// Section dimension (Rectangular)
double youngModulus{25e9}; // Module Young
double b{0.02}, h{0.05};
```

```

// External Force applying
double forceZ{-1.5e6};
Vector<Vector<double>> externalForce{
    {0.0, 0.0, 0.0},    // Node 0: no force
    {0.0, 0.0, 0.0},    // Node 1: no force
    {0.0, 0.0, forceZ}, // Node 2: 1.5 MN downward (z-direction)
    {0.0, 0.0, forceZ}, // Node 3: 1.5 MN downward
    {0.0, 0.0, 0.0},    // Node 4: no force
    {0.0, 0.0, 0.0},    // Node 5: no force
    {0.0, 0.0, forceZ}, // Node 6: 1.5 MN downward
    {0.0, 0.0, forceZ}  // Node 7: 1.5 MN downward
};

// Choosing non-linear Saint-Venant Kirchhoff law
int law{1};

//Penalization method
double penalty{1.0 / 1e-18}; // 1/epsilon

// tolerance for Newton's method
double epsilon{1e-8};} // namespace modelParameters
```

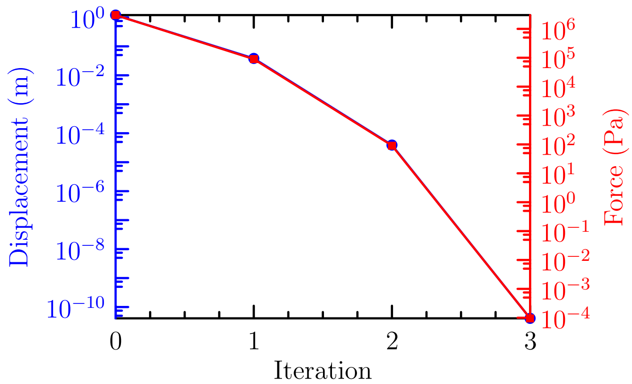
which produce this output to the console:

```
=== 3D TRUSS PROBLEM ===
Iteration 0: |Fk| = 3e+06: |dx| = 1.2
Iteration 1: |Fk| = 90000: |dx| = 0.0382979
Iteration 2: |Fk| = 91.6704: |dx| = 3.90883e-05
Iteration 3: |Fk| = 9.54932e-05: |dx| = 4.07184e-11
Solution converged at iteration 4.

Final displacement: [0, 0, 0, 0, 0, 0, 0, 0, -0.619168, 0, 0,
   -0.619168, 0, 0, 0, 0, 0, 0, 0, 0, -0.619168, 0, 0, -0.619168]

Final positions:
  Node 0: [0, 0, 0]
  Node 1: [10, 0, 0]
  Node 2: [0, 0, 9.38083]
  Node 3: [10, 0, 9.38083]
  Node 4: [0, -10, 0]
  Node 5: [10, -10, 0]
  Node 6: [0, -10, 9.38083]
  Node 7: [10, -10, 9.38083]

Reactions at supports:
  Node 0: [0, 0, 1.5e+06]
  Node 1: [0, 0, 1.5e+06]
  Node 4: [0, 0, 1.5e+06]
  Node 5: [0, 0, 1.5e+06]

Time elapsed: 0.785562 seconds
```

The code has also been checked with the previous 2D console problem, showing accurate result. After the 4^{th} iteration, both the displacement and the sum of external and internal forces (including reaction force) on all the nodes, converged to inferior of ϵ. Besides, the script verifies the stiffness matrix $\underline{\underline{\mathcal{K}}}$ before applying the kinematic boundary condition (via virtual field constraint) is symmetric and singular ($\det(\underline{\underline{\mathcal{K}}}) = 0$). Intuitively, the deformed schematic and the displacement countering the forces applied is in good accordance. The penalization method provided machine-error values of displacement, which is approximately equal to 0 on fixed nodes.



(a) Deformed truss schematic



(b) Magnitude sum of displacements and forces on whole set of nodes

Figure 6: Solutions of the 3D truss problem

3.3 FEM1D

Taking trivial values as follows: $f = 5$ kN, $F = 10$ kN, $EA = 10$ kN, $L = 1$ m, one can verify the analytic solution to be: $R = -15$ kN, $u(x) = 1.5\,x - 0.25\,x^2$ (m), $N(x) = 15 - 5x$ (kN).

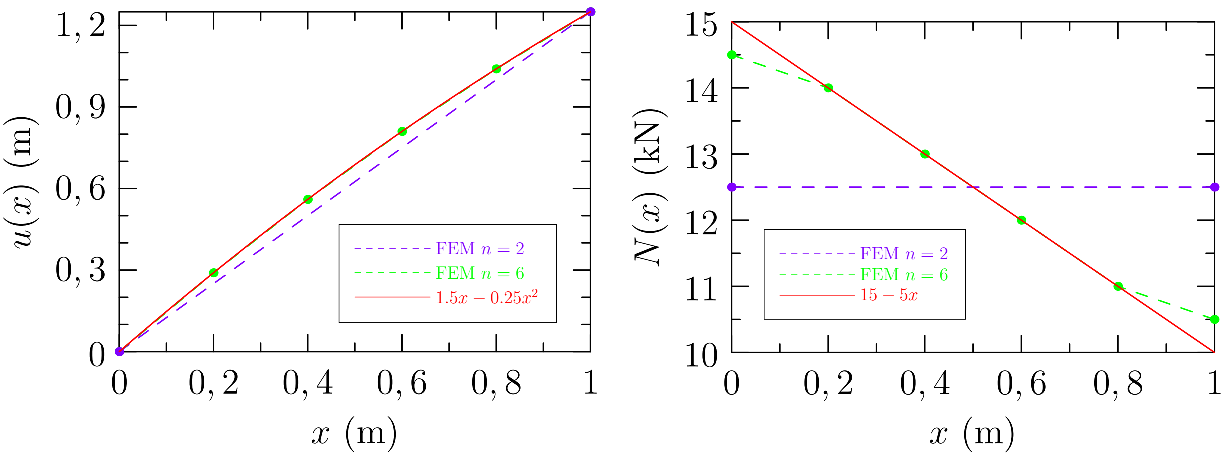
```
namespace modelParameters {
// Problem's domain (a,b)
double a{0.0};
double b{1.0};
// Dirichlet and Neumann boundary values
constexpr double g{0.0};  // Dirichlet at x=0 (u(0)=g)
constexpr double h{10.0}; // Neumann at x=1 (N(1)=h=10kN)
constexpr double uniform_load{5.0}; // Uniform distributed load
constexpr Index nNodes{6};                // Numbers of nodes
constexpr Index nElements{nNodes - 1}; // Numbers of elements
constexpr double EA = 10.0;}
```

Giving the output to the console:

```
=== FEM 1D PROBLEM ===
Coordinates of nodes x_i:
[0, 0.2, 0.4, 0.6, 0.8, 1]
Displacement vector U:
[0, 0.29, 0.56, 0.81, 1.04, 1.25]
```

```
Reaction force vector R:
[-15, -0, -4.44089e-16, 2.22045e-16, 2.22045e-16, -7.10543e-15]
Axial force vector N:
[14.5, 13.5, 12.5, 11.5, 10.5, 10.5]
Time elapsed: 0.000358521 seconds
```

Using the script, though the reaction force at the first node $R(x = 0) = -15\text{kN}$ is always calculated precisely, the displacement function $u(x)$ and the axial force $N(x)$ are highly depended on the number of nodes. Figure 7 shows the numerical solution of the code, comparing with the analytical one. It was shown that the displacement $u(x)$ is obviously more precise, when the bar is discretized by using more elements. The particular point of `FEM` comparing with Galerkin's method is using special shape function, particularly in this case is first-order hat function, via Lagrange's basis. This is well explained by when we discretized $u(x) = N_j u_j$. As u_j is a constant, $u(x)$ is a first order polynomial (fig. 7a), leading to its derivative to be a constant, consequently $N(x) = EA\ du/dx$ is a constant on each element also (fig. 7b). Though this phenomenon is expected, as in the appendix A of [1].



(a) Displacement

(b) Axial Force

Figure 7: Solutions of the 1D FEM problem

3.4 2D Finite Element Method

Under construction... Currently stuck at mesh generation.

4 Application & Conclusion

4.1 Console and Truss

The code is able to handle any non-linear truss problem, 2D and 3D, including the console problem above. Numbers of bars, their directions and identically, components of forces and kinematic conditions could be implemented according to the problem in need. Though the Hardening incremented law should be implemented in the future to wider the range of problems that could be handled.

4.2 1D Finite Element Method

The code has been implemented in a way to solve any 1D bar problem, with whatever the constraint conditions are, both Dirichlet and Neuman's condition. As long as the differential equation satisfied the form $u_{xx} = f(x)$ with $f(x)$ is a polynomial function of x, including cx^0. This leads to the application of distributed, concentrated and/or spatially varying load exhibited on the bar, manipulated by the two function `rhsFunction() & applyBC()`. Though the numerical cost could be reduced strongly using reference element, element Matrix, ... etc but this initial version is to test out all the functioning of the sub-coded also. Gladly, the solution seems to adapted well with the analytical one as expected.

4.3 Conclusion

In a nutshell, the code provided trustable result. Further, implemented still rested to continue. Thanks to using C++, time elapsed to execute one code is less than 1 second on the student's computer. Further optimization will continue to be implemented.

A Appendix

Full version of the codes, including libraries used, could be downloaded in this link. The scripts presented in this report are `console2D.cpp, truss3D.cpp, fem1D.cpp`, along with their compiled programs. Figures and graphs are fully provided in the `report` respiratory.

References

[1] DAL-PONT, S. (2020). *Numerical Methods for Non-Linear Mechanics.* Université Grenoble Alpes, Grenoble, France. 1, 12